



UNIVERSIDAD DEL ISTMO
FACULTAD DE INGENIERÍA

Comentado [ALCR1]: Didvin:
Para hacer las correcciones, por favor sigue las instrucciones listadas a continuación:

1. Trabaja las correcciones sobre este mismo documento.
2. Por favor no borres mis comentarios así será más fácil ubicar las correcciones y mi trabajo será más eficiente.
3. Cuando me devuelvas el archivo, sólo cámbiale la fecha

ANTEPROYECTO
AUDITORÍA AUTOMATIZADA DE VULNERABILIDADES OWASP EN DJANGO
REST CON RAG

DIDVIN NOHEL ESTRADA PINEDA

Guatemala, 3 de agosto de 2026

Comentado [ALCR2]: ¿fecha?

INTRODUCCIÓN

El crecimiento exponencial de las interfaces de programación de aplicaciones de tipo transferencia de estado representacional (*REST*) ha transformado la arquitectura de los sistemas de información modernos. Investigaciones recientes evidencian que las interfaces de programación de aplicaciones *REST* se han convertido en el principal vector de ataque en los ecosistemas digitales modernos: Atlidakis et al. (2019) demostraron la presencia sistemática de vulnerabilidades explotables en API *REST* de producción mediante pruebas automatizadas, y Zhou et al. (2019) identificaron que los patrones de vulnerabilidad en aplicaciones Python son estructuralmente recurrentes y detectables. En este contexto, Django *REST Framework* —la biblioteca de desarrollo de *software* de Python más utilizada para la construcción de interfaces de programación de aplicaciones— acumula más de un millón de descargas mensuales en el repositorio PyPI, lo que refleja su prevalencia en entornos de producción a nivel mundial.

A pesar de esta adopción masiva, las herramientas de análisis de seguridad de uso general disponibles en la actualidad —entre ellas Bandit, Semgrep y OWASP ZAP— no comprenden los patrones internos de Django *REST Framework*. En consecuencia, el desarrollador carece de retroalimentación contextualizada y oportuna durante las fases tempranas del ciclo de desarrollo, lo que favorece la introducción inadvertida de vulnerabilidades clasificadas en el OWASP API Security Top 10 (2023). Esta ausencia de herramientas especializadas constituye el problema central que motiva el presente trabajo de investigación.

El objetivo general del presente trabajo consiste en desarrollar una herramienta de interfaz de línea de comandos (*CLI*) que automatice la auditoría de seguridad de interfaces de programación de aplicaciones desarrolladas con Django *REST Framework*, detectando vulnerabilidades de seis categorías del OWASP API Security Top 10 (2023) mediante análisis estático basado en Árboles de Sintaxis Abstracta y análisis dinámico mediante sondas HTTP, y generando recomendaciones de remediación contextualizadas a través de un *pipeline* de Generación Aumentada por Recuperación con ejecución completamente local. La justificación de este trabajo radica en la necesidad de reducir el tiempo entre la introducción de una vulnerabilidad y su detección, sin depender de servicios externos ni de

Comentado [ALCR3]: La sangría debe ser de 5 espacios, como esta que te dejo aquí de ejemplo. Corrígelas en todo el trabajo.

conectividad a la red, lo que lo hace viable en entornos restringidos y con código fuente propietario.

El método adoptado combina análisis estático de código fuente —mediante inspección de Árboles de Sintaxis Abstracta con el módulo estándar *ast* de Python— con análisis dinámico mediante sondas HTTP que interactúan con una instancia en ejecución del proyecto auditado. Las recomendaciones de remediación se generan a través de un *pipeline* de Generación Aumentada por Recuperación que consulta una base de conocimiento local indexada en ChromaDB y utiliza el modelo Mistral 7B ejecutado mediante Ollama. La herramienta es evaluada con métricas de precisión y *recall* en un entorno controlado y en tres proyectos externos de código abierto.

La motivación personal del autor surge de la observación directa, durante el ejercicio profesional en el ecosistema de desarrollo Python en Guatemala, de vulnerabilidades recurrentes en interfaces de programación de aplicaciones construidas con Django REST *Framework*: serializadores con *fields = '__all__'* que exponen campos de contraseña en las respuestas JSON, vistas sin *permission_classes* que permiten acceso sin autenticación, y configuraciones *DEBUG = True* desplegadas inadvertidamente en entornos de producción. Estas observaciones evidenciaron que el problema no es de conocimiento —los desarrolladores conocen las buenas prácticas— sino de retroalimentación oportuna: en ausencia de herramientas especializadas, la verificación de seguridad ocurre tarde, generalmente mediante revisiones de código manuales o después de incidentes en producción. El presente trabajo busca sistematizar esa retroalimentación en una herramienta automatizada, reproducible y local.

1. MARCO DE REFERENCIA

1.1 ANTECEDENTES

Las interfaces de programación de aplicaciones han experimentado un proceso de estandarización que comenzó con la adopción generalizada del estilo arquitectónico REST, definido formalmente por Roy Fielding en su disertación doctoral del año 2000. *Representational State Transfer* establece seis restricciones fundamentales —cliente-servidor, sin estado, caché, interfaz uniforme, sistema en capas y código bajo demanda— que estructuran la forma en que los sistemas de información intercambian representaciones de recursos a través de HTTP.¹

La preocupación por la seguridad de las interfaces de programación de aplicaciones se formalizó con la fundación de OWASP (Open Worldwide Application Security Project) en 2001, organización sin fines de lucro dedicada a mejorar la seguridad del *software*. OWASP publicó su primera lista específica de vulnerabilidades en interfaces de programación de aplicaciones en 2019 y la actualizó en 2023, estableciendo el OWASP API Security Top 10 como referencia estándar de la industria para la clasificación y priorización de riesgos en este tipo de componentes.²

En el campo del análisis estático de *software*, la herramienta Bandit fue desarrollada por el equipo PyCQA para detectar problemas de seguridad comunes en código Python mediante reglas predefinidas. Su conjunto de reglas cubre principalmente credenciales incluidas directamente en el código fuente (reglas B105 y B106) e inyección de comandos del sistema operativo, pero no incluye patrones específicos de Django REST *Framework* ni de las categorías del OWASP API Security Top 10. En forma paralela, Semgrep surgió como motor de análisis estático de patrones multi-lenguaje; su conjunto de reglas *p/django* cubre inyección SQL y protección CSRF, sin abordar los patrones de autenticación y autorización internos de Django REST *Framework*. Chess y McGraw (2004) establecieron que el análisis

¹FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Tesis doctoral. University of California, Irvine, 2000. [Consulta: 23 de julio de 2026] Disponible en: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

²OWASP Foundation. OWASP API Security Top 10 2023 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://owasp.org/API-Security/>

estático permite detectar clases de problemas estructurales que escapan a la revisión manual de código.³

Los primeros trabajos que vinculan modelos de lenguaje de gran escala con la seguridad del *software* datan del período 2021-2023. Pearce et al. (2023) evaluaron la capacidad de GPT-4 para reparar vulnerabilidades de forma automática, demostrando que los modelos pueden proponer correcciones coherentes, aunque con una tasa significativa de alucinaciones cuando carecen de documentación de referencia actualizada. Este hallazgo constituyó el antecedente directo que motivó la incorporación de un *pipeline* de Generación Aumentada por Recuperación en la arquitectura del presente trabajo.⁴

El paradigma de Generación Aumentada por Recuperación fue formalizado por Lewis et al. (2020) en el trabajo «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks», que demostró cómo la combinación de un recuperador de información con un modelo generativo mejora la precisión factual de las respuestas en tareas que requieren conocimiento especializado. Este trabajo constituye la base teórica del componente RAG implementado en la presente investigación.⁵

La relevancia de Reimers y Gurevych (2019) como antecedente reside en la viabilidad computacional que Sentence-BERT aportó a los sistemas RAG. Antes de esta contribución, la generación de representaciones vectoriales de calidad requería modelos de tamaño considerable con alto costo de inferencia, lo que hacía inviable la ejecución local en tiempo real. El modelo *all-MiniLM-L6-v2* —derivado de la arquitectura Sentence-BERT— puede procesar cientos de fragmentos de texto por segundo en CPU, eliminando la necesidad de hardware especializado para la fase de indexación y recuperación del *pipeline* RAG. Esta característica es indispensable para el objetivo de operación completamente local de la herramienta.⁶

³CHESS, Brian y McGRAW, Gary. Static Analysis for Security. En: IEEE Security & Privacy, vol. 2, núm. 6, 2004. pp. 76-79. [Consulta: 23 de julio de 2026] Disponible en: <https://ieeexplore.ieee.org/document/1386725>

⁴PEARCE, Hammond, et al. Examining Zero-Shot Vulnerability Repair with Large Language Models. En: Proceedings of IEEE S&P, 2023. pp. 2339-2356. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2112.02125>

⁵LEWIS, Patrick, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. En: Advances in Neural Information Processing Systems, vol. 33, 2020. pp. 9459-9474. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2005.11401>

⁶REIMERS, Nils y GUREVYCH, Iryna. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. En: Proceedings of EMNLP, 2019. pp. 3982-3992. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1908.10084>

Un antecedente de especial relevancia en el ámbito de la cuantización de modelos de lenguaje es el trabajo de Jiang et al. (2023) que introdujo el modelo Mistral 7B. Este modelo demostró que una arquitectura de siete mil millones de parámetros, con técnicas de atención de ventana deslizante y atención de consulta agrupada, puede alcanzar un rendimiento comparable al de modelos de trece mil millones de parámetros de la generación anterior.⁷

En su variante cuantizada Q4_K_M, Mistral 7B requiere aproximadamente cuatro gigabytes de memoria RAM para su ejecución, lo que lo hace compatible con el *hardware* disponible en estaciones de trabajo de desarrollo típicas. La herramienta Ollama facilita la gestión de modelos cuantizados mediante una interfaz REST local, eliminando la complejidad de la configuración directa del *runtime* de inferencia y permitiendo que la herramienta de auditoría interactúe con el modelo mediante llamadas HTTP estándar.⁸

En el contexto centroamericano, la literatura sobre herramientas de seguridad para el desarrollo de software adaptadas a entornos con recursos limitados es escasa. La mayoría de los estudios sobre análisis de vulnerabilidades en *frameworks web* provienen de grupos de investigación de países con mayor inversión en seguridad informática. Sin embargo, la creciente adopción de servicios digitales en Guatemala y la región —en particular en los sectores de banca, telecomunicaciones y gobierno electrónico— hace urgente el desarrollo de capacidades locales en esta área. El presente trabajo contribuye a llenar esta brecha al producir una herramienta de código abierto, documentada en español, y evaluada con metodología académica reproducible, que puede ser adoptada directamente por equipos de desarrollo de la región sin requerir infraestructura de nube ni licencias de herramientas comerciales.

1.2 SITUACIÓN ACTUAL

Django REST *Framework* es, en la actualidad, la solución más utilizada en el ecosistema Python para la construcción de interfaces de programación de aplicaciones de tipo REST. Con más de un millón de descargas mensuales en el repositorio PyPI, es adoptado por organizaciones de relevancia global en sectores que incluyen comercio electrónico, medios

Comentado [ALCR4]: No es correcto poner doble cita. Hay que colocarlas individualmente. Corrige todas las que tengas así en el trabajo

⁷JIANG, Albert Q., et al. Mistral 7B. arXiv:2310.06825 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2310.06825>

⁸TOUVRON, Hugo, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2307.09288>

digitales y servicios financieros. Su popularidad implica que los patrones de configuración inseguros que esta biblioteca permite —y que no son detectados por las herramientas existentes— tienen un impacto potencial de alcance masivo.

La evaluación realizada durante el desarrollo de la presente investigación evidenció una brecha crítica en las capacidades de las herramientas de análisis de seguridad disponibles: al ejecutar Bandit versión 1.9 contra una interfaz de programación de aplicaciones de prueba con veintiséis instancias de vulnerabilidades plantadas en seis categorías del OWASP API Security Top 10, la herramienta reportó exclusivamente cuatro hallazgos de severidad LOW correspondientes a credenciales incluidas directamente en el código fuente, con cobertura nula para las seis categorías objetivo. El conjunto de reglas *p/django* de Semgrep no produjo ningún hallazgo relevante. OWASP ZAP, por operar exclusivamente a nivel HTTP sin acceso al código fuente, tampoco detectó los patrones de configuración internos de Django REST Framework.⁹

Viglianisi et al. (2020) y Atlidakis et al. (2019) confirmaron limitaciones equivalentes en herramientas de pruebas de caja negra para API REST, que tampoco inspeccionan el código fuente.¹⁰

En el ámbito de la inteligencia artificial aplicada a la seguridad, la tendencia dominante consiste en incorporar modelos de lenguaje de gran escala en ciclos DevSecOps para la generación automática de recomendaciones de remediación. Sin embargo, las soluciones disponibles en el mercado operan mediante llamadas a interfaces de programación de aplicaciones en la nube, lo que presenta tres limitaciones significativas: costo operativo por *token*, transmisión de código fuente propietario hacia servidores externos, e incompatibilidad con entornos sin acceso a internet o con regulaciones estrictas de privacidad de datos.¹¹

La regulación de privacidad de datos en América Latina ha evolucionado significativamente en los últimos años. Guatemala cuenta con la Ley de Acceso a la Información Pública (Decreto 57-2008), y múltiples países de la región han adoptado regulaciones análogas al Reglamento General de Protección de Datos (GDPR) europeo. Estas

⁹VIGLIANISI, Emanuele, et al. RESTTESTGEN: Automated Black-Box Testing of RESTful APIs. En: Proceedings of IEEE ICST, 2020. pp. 367-377. [Consulta: 23 de julio de 2026] Disponible en: <https://ieeexplore.ieee.org/document/9159077>

¹⁰ATLIDAKIS, Vaggelis, et al. RESTler: Stateful REST API Fuzzing. En: Proceedings of ICSE, 2019. pp. 748-758. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1806.09099>

¹¹TOUVRON, Hugo, et al., Op. Cit.

regulaciones imponen restricciones sobre la transmisión de datos personales a servidores externos, lo que afecta directamente a las herramientas de análisis de seguridad basadas en la nube: el código fuente de un sistema de salud, educación o finanzas puede contener datos que bajo estas regulaciones no pueden transmitirse a un servicio externo para su análisis. Una herramienta que opera completamente de forma local elimina este obstáculo regulatorio, habilitando la auditoría automatizada en sectores de alta sensibilidad de datos.

La convergencia de estas limitaciones —ausencia de herramientas de análisis especializadas para Django REST *Framework*, dependencia de servicios externos para la generación asistida por inteligencia artificial, y restricciones regulatorias sobre la transmisión de código fuente— define la situación actual que el presente trabajo busca transformar mediante el desarrollo de una herramienta de auditoría híbrida, local y trazable a documentación oficial. La herramienta es especialmente relevante en el contexto de las organizaciones centroamericanas que carecen de infraestructura de seguridad especializada pero utilizan Django REST *Framework* para construir sistemas críticos.

En términos de madurez tecnológica, la disponibilidad de herramientas como Ollama y ChromaDB en versiones estables representa una ventana de oportunidad para implementar sistemas RAG locales en entornos de producción.¹²

Antes de 2023, la ejecución de modelos de lenguaje de calidad suficiente para generación de recomendaciones técnicas requería acceso a clústeres de GPU o a servicios en la nube, lo que hacía económicamente inviable su adopción en herramientas de desarrollo para equipos pequeños. La combinación de cuantización de modelos (Q4_K_M), arquitecturas de *transformer* optimizadas para CPU (Mistral 7B) y bases de datos vectoriales embebibles (ChromaDB) ha reducido los requisitos de *hardware* a niveles accesibles para estaciones de trabajo de desarrollo estándar, habilitando la categoría de herramientas de análisis de seguridad aumentadas por inteligencia artificial que el presente trabajo explora.¹³

1.3 MARCO TEÓRICO

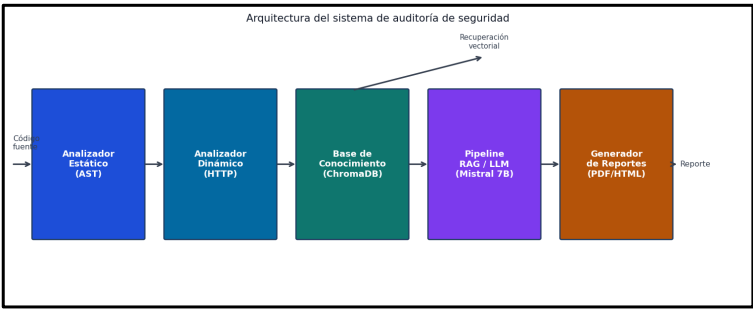
¹²JIANG, Albert Q., et al., Op. Cit.

¹³KARPUKHIN, Vladimir, et al. Dense Passage Retrieval for Open-Domain Question Answering. En: Proceedings of EMNLP, 2020. pp. 6769-6781. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2004.04906>

El sistema propuesto se articula en cinco módulos funcionales que operan de forma coordinada. El flujo de procesamiento inicia con el análisis del código fuente mediante el módulo de análisis estático basado en árboles de sintaxis abstracta (AST); continúa con el análisis dinámico, que envía solicitudes HTTP a la instancia en ejecución; y converge en el *pipeline* de generación aumentada por recuperación (RAG), el cual consulta la base de conocimiento vectorial para enriquecer el contexto del modelo de lenguaje local antes de generar las recomendaciones de remediación. El módulo generador de reportes consolida los hallazgos de ambos analizadores y las recomendaciones producidas por el modelo de lenguaje de gran tamaño (LLM), y genera el informe final en formato PDF o HTML. Todos los componentes se ejecutan localmente, sin depender de servicios externos ni transferir el código fuente a terceros.

Figura No. 1

ARQUITECTURA DEL SISTEMA DE AUDITORÍA DE SEGURIDAD



Fuente: Propia.

1.3.1 Seguridad de aplicaciones web y API REST

La seguridad de las aplicaciones web abarca el conjunto de controles, prácticas y tecnologías destinados a proteger los sistemas que intercambian información a través de protocolos de internet. En el contexto de las interfaces de programación de aplicaciones de tipo REST, la superficie de ataque difiere de la de las aplicaciones web tradicionales en aspectos estructurales que determinan el tipo de vulnerabilidades más frecuentes.¹⁴

Las aplicaciones web tradicionales operan bajo un paradigma orientado al documento: el servidor genera HTML completo que el navegador renderiza, y el estado de la sesión reside

Comentado [ALCR5]: No negrilla

Debes colocarle un recuadro a la figura.

Y lo más importante, debes escribir una introducción antes de la figura, por lo menos para explicar por qué está allí

Comentado [ALCR6]: Las palabras que escriba en rojo no van con mayúsculas

Las palabras resaltadas en verde van con cursiva. Corregir todas las web y otras que aparezcan por allí

Los acrónimos no tienen plural. Corregir todas las APIs que haya.

¹⁴CHESS, Brian y McGRAW, Gary, Op. Cit.

en el servidor mediante cookies. Las interfaces de programación de aplicaciones REST rompen este paradigma al separar por completo la lógica de presentación de la lógica de negocio, exponiendo recursos de datos estructurados que múltiples tipos de clientes — aplicaciones móviles, *frontends* de página única, sistemas de terceros— consumen directamente. Esta separación tiene implicaciones de seguridad fundamentales: el servidor ya no controla la presentación de los datos, y cualquier dato que devuelve puede ser leído, almacenado o reenviado por el cliente con total autonomía.

Viglianisi et al. (2020) identificaron que los ataques contra interfaces de programación de aplicaciones REST se concentran en los puntos donde el sistema acepta **entrada** del usuario, procesa datos o toma decisiones de autorización. En dichas interfaces, estas tres zonas coinciden en cada *endpoint* expuesto: el identificador de recurso en la URL constituye entrada del usuario, el cuerpo JSON de la petición contiene datos a procesar, y los encabezados de autorización determinan las decisiones de acceso. La multiplicación de *endpoints* y la variedad de clientes que los consumen amplifican la superficie de ataque respecto de las aplicaciones web monolíticas.¹⁵

1.3.1.1 El paradigma REST

Fielding definió en su disertación doctoral (2000) seis restricciones arquitectónicas que caracterizan el estilo REST: (1) separación cliente-servidor, (2) comunicación sin estado, (3) respuestas *cacheables*, (4) interfaz uniforme basada en recursos e identificadores URI, (5) sistema en capas que puede incluir intermediarios, y (6) código bajo demanda de forma opcional. Estas restricciones eliminan la necesidad de que el servidor mantenga estado de sesión, lo que facilita la escalabilidad pero delega en el cliente la responsabilidad de incluir credenciales en cada petición, ampliando la superficie de ataque.¹⁶

La restricción de interfaz uniforme es la que más impacto tiene sobre la seguridad. Al exigir que cada recurso sea identificable mediante un URI único y que las operaciones sobre él se expresen mediante verbos HTTP estándar (GET, POST, PUT, PATCH, DELETE), REST crea una estructura predecible que facilita tanto el consumo legítimo como el abuso malicioso. Un atacante que comprende la estructura de **URI** de una interfaz de programación

Comentado [ALCR7]: Busca en el manual cómo debes citar cuando repites una fuente

Yo te marqué algunas pero tienes varias que citas varias veces. Hay que corregir las citas al pie entonces.

Comentado [ALCR8]: esta tampoco se pone en plural.

¹⁵VIGLIANISI, Emanuele, et al., Op. Cit.

¹⁶FIELDING, Roy Thomas, Op. Cit.

de aplicaciones REST puede intentar enumerar recursos incrementando identificadores numéricos, descubrir *endpoints* no documentados mediante fuerza bruta de rutas, o inferir la estructura del modelo de datos a partir de los campos devueltos en las respuestas.

1.3.1.2 API REST como superficie de ataque

A diferencia de las aplicaciones web que renderizan HTML en el servidor, las interfaces de programación de aplicaciones REST exponen directamente recursos de datos en formato JSON o XML, con autenticación basada en *tokens* y mayor autonomía del cliente para determinar qué campos enviar o modificar. Esta autonomía introduce categorías de vulnerabilidades específicas: acceso no autorizado a objetos por manipulación de identificadores en la ruta URL (BOLA), modificación masiva de propiedades mediante el envío de campos no declarados (asignación masiva), y acceso a funciones administrativas sin verificación de rol. Chess y McGraw (2004) definen la superficie de ataque como el conjunto de puntos de entrada a través de los cuales un atacante puede intentar introducir datos o extraer información del sistema.¹⁷

La naturaleza stateless de REST implica que cada petición debe contener toda la información necesaria para ser procesada, incluyendo las credenciales de autenticación. Esto se implementa típicamente mediante *tokens* JWT (JSON Web Tokens) o claves de API incluidas en encabezados HTTP. Si estos mecanismos se implementan incorrectamente —por ejemplo, omitiendo la validación del *token* en algunas vistas, o usando algoritmos de firma débiles— la consecuencia es el acceso no autorizado a recursos que deberían ser protegidos. La diferencia respecto de las aplicaciones web tradicionales es que en REST no existe un mecanismo de sesión del lado del servidor que pueda actuar como segunda línea de defensa: la validación del *token* en cada *endpoint* es la única barrera entre el cliente y los datos.

1.3.1.3 Estadísticas y tendencias de vulnerabilidades en API

La investigación académica sobre vulnerabilidades en interfaces de programación de aplicaciones REST evidencia una creciente preocupación por su seguridad. Atlidakis et al. (2019) demostraron mediante pruebas de caja negra sobre servicios REST reales que una

¹⁷CHESS, Brian y McGRAW, Gary, Op. Cit.

proporción significativa de interfaces presenta vulnerabilidades explotables automáticamente.¹⁸

Zhou et al. (2019), mediante análisis de grafos de flujo de código sobre proyectos de código abierto, identificaron que los patrones de vulnerabilidad en aplicaciones Python son estructuralmente recurrentes y detectables mediante análisis estático, lo que fundamenta la viabilidad del enfoque del presente trabajo.¹⁹

Entre los incidentes de seguridad más representativos relacionados con vulnerabilidades en interfaces de programación de aplicaciones REST se encuentran la filtración de datos de la plataforma T-Mobile (2021), que expuso información de más de cincuenta millones de usuarios mediante la explotación de una vulnerabilidad de tipo BOLA en una interfaz de programación de aplicaciones interna, y la brecha de seguridad de Peloton (2021), en la que datos de perfil de todos los usuarios de la plataforma eran accesibles sin autenticación a través de su interfaz de programación de aplicaciones pública. Ambos incidentes ilustran que las vulnerabilidades del OWASP API Security Top 10 no son únicamente teóricas, sino que producen consecuencias económicas y reputacionales de escala masiva en el ecosistema de producción real.²⁰

1.3.2 OWASP API Security Top 10 (2023)

OWASP (Open Worldwide Application Security Project) es una organización sin fines de lucro fundada en 2001 cuya misión consiste en mejorar la seguridad del *software* mediante la producción de metodologías, documentación y herramientas de uso libre. Su lista API Security Top 10 constituye el estándar de referencia más utilizado en la industria para la clasificación y priorización de riesgos en interfaces de programación de aplicaciones REST. La edición 2023 identifica diez categorías de vulnerabilidades; el presente trabajo aborda seis de ellas.²¹

La metodología de construcción del OWASP API Security Top 10 se basa en la recopilación de datos de incidentes reales reportados por la industria, el análisis de bases de

¹⁸ATLIDAKIS, Vaggelis, et al., Op. Cit.

¹⁹ZHOU, Yaqin, et al. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. En: Advances in Neural Information Processing Systems, vol. 32, 2019. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1909.03496>

²⁰OWASP Foundation, Op. Cit.

²¹Ibid.

datos de vulnerabilidades públicas como la NVD del NIST, y la consulta con expertos de seguridad de organizaciones de referencia mundial. La edición 2023 incorporó cambios significativos respecto de la versión 2019: la categoría API3 fue reformulada de «Exposición Excesiva de Datos» a «Autorización a Nivel de Propiedad de Objeto Deficiente», reconociendo que el problema no es únicamente la exposición sino también la modificación no autorizada de propiedades sensibles. La categoría API8 unificó las configuraciones inseguras de seguridad bajo un concepto más amplio que incluye encabezados HTTP incorrectos, modos de depuración activos y configuraciones de CORS permisivas.²²

Para el presente trabajo, la selección de las seis categorías a cubrir se realizó con base en dos criterios: (1) la existencia de un indicador de riesgo detectable en el código fuente de Django REST *Framework* mediante análisis estático de Árboles de Sintaxis Abstracta o mediante sonda HTTP, y (2) la frecuencia documentada de cada categoría en proyectos Django REST *Framework* según el análisis de CVE publicados en la NVD entre 2020 y 2024. Las categorías API6 (Acceso No Restringido a Flujos de Negocios Sensibles), API7 (Falsificación de Petición del Lado del Servidor), API9 (Gestión de Inventario Inadecuada) y API10 (Consumo No Seguro de API) quedaron fuera del alcance porque requieren análisis de lógica de negocio específica o inspección de flujo de datos interprocedural, capacidades que exceden el alcance de una herramienta de análisis estático general.

1.3.2.1 API1:2023 — *Broken Object Level Authorization* (BOLA)

Esta categoría —conocida también como IDOR (*Insecure Direct Object Reference*)— ocurre cuando el servidor no verifica que el usuario autenticado tiene derecho a acceder al objeto específico identificado en la ruta URL. El escenario típico consiste en que el usuario A accede a `/api/orders/1/` siendo que ese pedido pertenece al usuario B. En Django REST *Framework*, el indicador de riesgo detectado por la herramienta consiste en parámetros numéricos en la ruta URL sin filtrado explícito por `request.user` en el método `get_queryset()`. El impacto es la exposición masiva de datos de múltiples usuarios mediante enumeración secuencial de identificadores.²³

²²Ibid.

²³Ibid.

1.3.2.2 API2:2023 — *Broken Authentication*

Esta categoría engloba mecanismos de autenticación implementados incorrectamente. En Django REST *Framework*, la configuración de *permission_classes* determina qué clase de cliente puede acceder a cada vista. La ausencia de este atributo en una vista, cuando no existe una configuración global explícita de *DEFAULT_PERMISSION_CLASSES* en *settings.py*, resulta en que la vista aplique el permiso por defecto *AllowAny*, permitiendo el acceso sin autenticación. Esta situación es frecuente en proyectos que crecen sin una política de permisos centralizada.²⁴

1.3.2.3 API3:2023 — *Broken Object Property Level Authorization*

Esta categoría cubre la exposición o modificación no autorizada de propiedades individuales de los objetos. En Django REST *Framework*, los *ModelSerializer* con *fields = '__all__'* exponen todos los campos del modelo sin evaluación de su sensibilidad. Los campos *password*, *is_admin* o identificadores internos sin marcar como *write_only* o *read_only* representan patrones de riesgo concretos que la herramienta detecta.²⁵

1.3.2.4 API4:2023 — *Unrestricted Resource Consumption*

La ausencia de límites en el consumo de recursos de la interfaz de programación de aplicaciones permite ataques de fuerza bruta y denegación de servicio. En Django REST *Framework*, la protección nativa se implementa mediante *DEFAULT_THROTTLE_CLASSES* y *DEFAULT_THROTTLE_RATES* en la configuración de Django REST *Framework* del archivo *settings.py*. La ausencia de esta configuración constituye el indicador de riesgo detectado por el analizador estático.²⁶

1.3.2.5 API5:2023 — *Broken Function Level Authorization*

Esta categoría ocurre cuando los *endpoints* administrativos o de mayor privilegio son accesibles sin verificación adecuada del rol del usuario. La herramienta detecta este patrón dinámicamente enviando peticiones como usuario con privilegios estándar a rutas que

²⁴Ibid.

²⁵Ibid.

²⁶Ibid.

contienen indicadores administrativos en la URL, y verificando si la respuesta HTTP indica acceso concedido.²⁷

1.3.2.6 API8:2023 — **Security Misconfiguration**

Las configuraciones inseguras por defecto constituyen la categoría más frecuente en proyectos Django. La más crítica es *DEBUG = True* en entornos de producción: cuando Django opera en modo depuración, cada excepción no manejada expone en la respuesta HTTP el *stack trace* completo, los valores de las variables locales en cada cuadro de la pila, todas las consultas SQL ejecutadas en la petición y las rutas internas del sistema de archivos del servidor. Esta configuración constituye una fuente de información crítica para un atacante.²⁸

Es importante destacar que las categorías anteriores del OWASP API Security Top 10 no son vulnerabilidades mutuamente excluyentes: en proyectos reales, la presencia de una categoría suele correlacionarse con la presencia de otras. Por ejemplo, un proyecto con *permission_classes* ausentes (API2) frecuentemente también exhibe *ModelSerializer* con *fields = '__all__'* (API3) y carece de *DEFAULT_THROTTLE_CLASSES* (API4), sugiriendo que la ausencia de una política de seguridad centralizada conduce a configuraciones inseguras generalizadas. Esta correlación justifica el diseño de una herramienta que aborde simultáneamente múltiples categorías en lugar de enfocarse en una sola.²⁹

Desde la perspectiva de los estándares de cumplimiento regulatorio, la alineación con el OWASP API Security Top 10 proporciona un lenguaje común que facilita la comunicación entre equipos de desarrollo, auditores de seguridad y responsables de gestión de riesgos. Las organizaciones que someten sus sistemas a certificaciones de seguridad como PCI-DSS o SOC 2 requieren evidencia de que sus interfaces de programación de aplicaciones han sido evaluadas contra categorías de vulnerabilidades reconocidas por la industria. Una herramienta que genera reportes estructurados y trazables a categorías OWASP específicas facilita la producción de dicha evidencia de forma automatizada y reproducible, reduciendo

²⁷Ibid.

²⁸Ibid.

²⁹Ibid.

el costo de las auditorías de cumplimiento y aumentando la frecuencia con que pueden realizarse.³⁰

1.3.3 Django REST Framework

Django es un *framework* de desarrollo web de alto nivel para Python, fundamentado en el principio DRY (*Don't Repeat Yourself*). Adoptado por organizaciones como Instagram, Pinterest y Disqus para servir miles de millones de peticiones diarias, proporciona un ORM (*Object-Relational Mapper*), un sistema de autenticación, un panel de administración y protecciones de seguridad activadas por defecto. Django REST *Framework* extiende Django con las abstracciones necesarias para construir interfaces de programación de aplicaciones REST: serializadores, vistas genéricas, conjuntos de vistas, enrutadores automáticos, sistemas de permisos, autenticación y limitación de tasa. Pearce et al. (2023) utilizaron proyectos Django REST Framework en sus evaluaciones, evidenciando la representatividad de este ecosistema en la investigación de vulnerabilidades.³¹

1.3.3.1 **Serializers** y exposición de campos

Los *serializers* son el componente de Django REST *Framework* responsable de convertir las instancias de modelos Django en tipos de datos nativos de Python —posteriormente en JSON— y de validar los datos entrantes. El *ModelSerializer* genera automáticamente los campos del *serializer* a partir del modelo. La declaración *fields = '__all__'* incluye todos los campos del modelo, lo que puede exponer datos sensibles como contraseñas hashadas, indicadores de rol o claves primarias internas. Los atributos *read_only=True* y *write_only=True* permiten controlar la dirección en que cada campo puede fluir. Li et al. (2018) demostraron que la exposición involuntaria de campos en serializadores es equivalente a los patrones de filtrado de datos que su sistema VulDeePecker clasifica como vulnerabilidades.³²

La arquitectura de Django REST *Framework* sigue el patrón de vista-serializador-modelo. El flujo de una petición HTTP entrante es el siguiente: el enrutador de Django distribuye la

³⁰CHESS, Brian y McGRAW, Gary, Op. Cit.

³¹PEARCE, Hammond, et al., Op. Cit.

³²LI, Zhen, et al. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. En: Proceedings of NDSS, 2018. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1801.01681>

petición a la vista correspondiente; la vista ejecuta los *middlewares* de autenticación y verifica los permisos; si la verificación es exitosa, llama al serializador con los datos de entrada; el serializador valida y deserializa; la vista interactúa con la capa de modelo mediante el ORM de Django; el serializador serializa el objeto de respuesta; y la vista retorna un objeto *Response* con el código HTTP y los datos serializados. Cada uno de estos pasos es un punto potencial de vulnerabilidad si no se configura adecuadamente.³³

La adopción de Django REST *Framework* por parte de organizaciones de escala industrial no está exenta de riesgos. Instagram, que basa parte de su infraestructura en Django, mantiene equipos dedicados de ingeniería de seguridad que auditan continuamente la configuración de sus interfaces de programación de aplicaciones. Sin embargo, la mayoría de las organizaciones que adoptan Django REST *Framework* son equipos pequeños o medianos que carecen de recursos dedicados a la seguridad de *software*. Para estas organizaciones, una herramienta automatizada de auditoría es la principal —y en muchos casos la única— línea de defensa proactiva.³⁴

1.3.3.2 Sistema de permisos y autenticación

El atributo *permission_classes* de cada vista determina qué clases de usuarios pueden acceder a ella. Las clases predefinidas incluyen *IsAuthenticated*, *IsAdminUser* y *AllowAny*. El valor por defecto de *DEFAULT_PERMISSION_CLASSES* en una instalación nueva de Django REST *Framework* es *AllowAny*, lo que significa que cualquier vista sin declaración explícita de permisos será accesible sin autenticación. La autenticación se configura mediante *DEFAULT_AUTHENTICATION_CLASSES* e incluye *TokenAuthentication*, *SessionAuthentication* y JWT mediante la biblioteca *django-rest-framework-simplejwt*. La seguridad de estos mecanismos de autenticación ha sido analizada por Pearce et al. (2023), quienes identificaron que su correcta configuración es determinante para prevenir accesos no autorizados.³⁵

La comprensión del sistema de permisos de Django REST *Framework* es fundamental para interpretar correctamente los hallazgos del analizador estático. Existen tres niveles de configuración de permisos: (1) configuración global mediante

³³CHESS, Brian y McGRAW, Gary, Op. Cit.

³⁴PEARCE, Hammond, et al., Op. Cit.

³⁵Ibid.

`DEFAULT_PERMISSION_CLASSES` en `REST_FRAMEWORK` dentro de `settings.py`, que aplica a todas las vistas que no declaran permisos explícitos; (2) configuración por vista mediante el atributo `permission_classes` en la clase de vista o mediante el decorador `@permission_classes` en vistas basadas en función; y (3) configuración por acción mediante el decorador `@action` en conjuntos de vistas. La herramienta desarrollada detecta la ausencia de configuración en los primeros dos niveles mediante análisis estático de Árboles de Sintaxis Abstracta, y verifica el comportamiento efectivo en el tercer nivel mediante sondas HTTP dinámicas.³⁶

1.3.4 Análisis **estático de código fuente**

El análisis estático de código fuente —también denominado SAST (*Static Application Security Testing*)— consiste en examinar el código fuente sin ejecutarlo, con el objetivo de identificar patrones que constituyan indicadores de riesgo de seguridad. A diferencia del análisis dinámico, el SAST puede integrarse en el ciclo de desarrollo sin necesidad de un entorno de ejecución activo.³⁷

1.3.4.1 Árboles de **sintaxis abstracta** (AST)

Un **árbol de sintaxis abstracta** (AST por su sigla en inglés) es la representación en forma de árbol de la estructura sintáctica del código fuente, en la que cada nodo del árbol corresponde a una construcción del lenguaje. El módulo estándar `ast` de Python proporciona las funciones `ast.parse()` para convertir código fuente en un árbol y `ast.walk()` para recorrerlo. Los nodos relevantes para el análisis de seguridad en Django REST *Framework* incluyen `ClassDef` (definiciones de clase), `Assign` (asignaciones) y `FunctionDef` (definiciones de función). La ventaja del análisis basado en AST frente al basado en expresiones regulares reside en su robustez ante variaciones de formato y en su precisión para identificar estructuras sintácticas completas. Li et al. (2018) emplearon representaciones de flujo de código similares a los AST para entrenar VulDeePecker, demostrando la eficacia de este tipo de representación estructural para la detección de vulnerabilidades.³⁸

³⁶CHESS, Brian y McGRAW, Gary, Op. Cit.

³⁷Ibid.

³⁸LI, Zhen, et al., Op. Cit.

Una expresión regular que busca la cadena «`permission_classes`» en el código fuente identificaría cualquier mención de esa cadena, independientemente de si aparece como atributo de clase, como argumento de función, como comentario o como valor de cadena en un diccionario. El *árbol de sintaxis abstracta*, en contraste, permite especificar exactamente: «busca definiciones de clase que hereden de *APIView* o de cualquier vista genérica de Django REST Framework, y que no tengan un atributo de clase denominado *permission_classes* asignado en el cuerpo de la clase». Esta especificidad reduce los falsos positivos y permite detectar el patrón de riesgo con precisión quirúrgica, incluso en proyectos con decenas de miles de líneas de código y múltiples archivos de vistas.

1.3.4.2 Indicadores de riesgo vs. vulnerabilidades confirmadas

El análisis estático identifica *security smells* o patrones frecuentemente asociados a vulnerabilidades, pero no puede confirmar la explotabilidad sin contexto de ejecución. La distinción entre un indicador de riesgo y una vulnerabilidad confirmada es fundamental: una vista sin *permission_classes* explícito puede ser segura si existe una configuración global que establece permisos restrictivos. Esta limitación estructural del análisis estático implica que los resultados deben interpretarse como puntos de inspección prioritarios, no como confirmaciones de explotabilidad. La tasa de falsos positivos se cuantifica mediante la métrica de precisión ($P = TP / (TP + FP)$).³⁹

Sistemas basados en aprendizaje profundo como SySeVR (Li et al., 2022) y LineVul (Fu y Tantithamthavorn, 2022) han demostrado que la combinación de representaciones estructurales del código con modelos de transformadores mejora la tasa de detección respecto de enfoques puramente basados en reglas.⁴⁰

³⁹LI, Zhen, et al. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. En: IEEE Transactions on Dependable and Secure Computing, vol. 19, núm. 4, 2022. pp. 2244-2258. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1807.06756>

⁴⁰FU, Michael y TANTITHAMTHAVORN, Chakkrit. LineVul: A Transformer-based Line-Level Vulnerability Prediction. En: Proceedings of MSR, 2022. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2205.02368>

Feng et al. (2020) introdujeron CodeBERT, un modelo preentrenado sobre código fuente y lenguaje natural que establece el estado del arte en tareas de comprensión de código, y que motiva el uso de representaciones semánticas en el pipeline RAG del presente trabajo.⁴¹

1.3.5 Análisis dinámico de seguridad

El análisis dinámico de seguridad —también denominado DAST (*Dynamic Application Security Testing*)— consiste en evaluar la seguridad de una aplicación mediante la interacción con ella en tiempo de ejecución. En el contexto de interfaces de programación de aplicaciones REST, el DAST implica el envío de peticiones HTTP diseñadas para verificar comportamientos que indiquen vulnerabilidades.⁴²

Atlidakis et al. (2019) y Viglianisi et al. (2020) propusieron generadores automáticos de casos de prueba para API REST basados en este principio, sentando las bases del análisis dinámico especializado en interfaces de este tipo.⁴³

1.3.5.1 HTTP **probing** y análisis de respuestas

Las sondas HTTP implementadas en el presente trabajo verifican: (1) acceso sin autenticación a *endpoints* que deberían requerirla; (2) acceso a recursos de otros usuarios mediante modificación del identificador en la URL; (3) aceptación de campos adicionales en el cuerpo de la petición (asignación masiva); (4) ausencia de respuesta HTTP 429 tras veinte peticiones consecutivas (indicador de ausencia de limitación de tasa); y (5) acceso sin verificación de rol a *endpoints* administrativos. Cada sonda documenta la condición que puede generar un falso positivo, dado que el comportamiento esperado puede depender de configuraciones específicas del proyecto.

1.3.5.2 Herramientas existentes: ZAP, Spectral y 42Crunch

OWASP ZAP es un proxy de intercepción HTTP que realiza escaneo activo y pasivo de aplicaciones web. Opera a nivel de protocolo HTTP sin acceso al código fuente, lo que le

⁴¹FENG, Zhangyin, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. En: Findings of EMNLP, 2020. pp. 1536-1547. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2002.08155>

⁴²ATLIDAKIS, Vaggelis, et al., Op. Cit.

⁴³VIGLIANISI, Emanuele, et al., Op. Cit.

impide detectar patrones de configuración internos de Django REST *Framework* como la ausencia de *permission_classes* o de *DEFAULT_THROTTLE_CLASSES*.⁴⁴

Spectral y 42Crunch analizan especificaciones OpenAPI en tiempo de diseño, validando la estructura declarada de la interfaz de programación de aplicaciones, pero requieren que el desarrollador mantenga una especificación actualizada y no tienen visibilidad del código de implementación. Atlidakis et al. (2019) identificaron que las herramientas basadas exclusivamente en especificaciones OpenAPI no detectan vulnerabilidades de implementación no declaradas en la especificación.⁴⁵

La complementariedad entre el análisis estático y el dinámico es una de las contribuciones arquitectónicas centrales de la herramienta. El análisis estático puede detectar la ausencia de *permission_classes* en el código fuente —un indicador de riesgo— pero no puede confirmar si esa ausencia resulta en una vulnerabilidad real, porque el comportamiento depende también de la configuración global de *settings.py*. El análisis dinámico, en cambio, verifica el comportamiento efectivo enviando una petición sin autenticación y observando si la respuesta es HTTP 200 (acceso concedido, posible vulnerabilidad) o HTTP 401/403 (acceso denegado, protección activa). La combinación de ambos análisis proporciona tanto cobertura amplia como confirmación empírica, reduciendo la dependencia del desarrollador de interpretar individualmente cada hallazgo.⁴⁶

Una limitación relevante del análisis dinámico en el contexto del presente trabajo es la necesidad de interpretar correctamente los códigos de respuesta HTTP. Un código 200 en respuesta a una petición sin autenticación no es automáticamente una vulnerabilidad: algunos *endpoints* legítimamente públicos devuelven 200. Un código 429 confirma que existe limitación de tasa, pero su ausencia no confirma explotabilidad porque el límite podría estar configurado en un proxy reverso externo no visible a la sonda. Por esta razón, la herramienta documenta explícitamente los supuestos de cada sonda: la sonda de autenticación asume que todos los *endpoints* evaluados deberían requerir autenticación; la sonda de BOLA asume que los identificadores numéricos en la URL son de objetos de un usuario diferente. Estos

⁴⁴ATLIDAKIS, Vaggelis, et al., Op. Cit.

⁴⁵VIGLIANISI, Emanuele, et al., Op. Cit.

⁴⁶CHESS, Brian y McGRAW, Gary, Op. Cit.

supuestos se explicitan en el reporte generado, permitiendo al desarrollador interpretar cada hallazgo en el contexto de la configuración real de su proyecto.⁴⁷

1.3.6 Modelos de lenguaje de gran escala aplicados a seguridad

Los **modelos de lenguaje de gran escala** (*Large Language Models*, LLM) son redes neuronales con decenas de miles de millones de parámetros entrenadas sobre corpus masivos de texto, cuya arquitectura base es el Transformer propuesto por Vaswani et al. (2017).⁴⁸

Su capacidad de generar texto coherente y contextualizado los hace candidatos naturales para tareas de síntesis de documentación de seguridad y generación de recomendaciones de remediación.⁴⁹

1.3.6.1 El problema de las alucinaciones

Una alucinación en el contexto de los LLM es la generación de información plausible pero factualmente incorrecta. Ji et al. (2023) realizaron la revisión sistemática más extensa sobre este fenómeno, documentando sus causas y mecanismos en modelos de lenguaje naturales.⁵⁰

Pearce et al. (2023) constataron que los LLM tienden a proponer paquetes inexistentes, citar URLs fabricadas o describir categorías OWASP con nombres incorrectos cuando carecen de documentación de referencia actualizada. En el contexto de recomendaciones de seguridad, una alucinación puede conducir al desarrollador a aplicar una corrección incorrecta o a adoptar una dependencia inexistente, erosionando la confianza en la herramienta.⁵¹

Los LLM evolucionaron a partir de la arquitectura Transformer (Vaswani et al., 2017) y los modelos de representación contextual como BERT (Devlin et al., 2019).⁵²

Brown et al. (2020) demostraron con GPT-3 que los LLM de escala suficiente pueden realizar tareas complejas con un número reducido de ejemplos en el *prompt* (*few-shot learning*), característica que fundamenta el diseño del *prompt* de enriquecimiento RAG

⁴⁷ATLIDAKIS, Vaggelis, et al., Op. Cit.

⁴⁸VASWANI, Ashish, et al. Attention Is All You Need. En: Advances in Neural Information Processing Systems, vol. 30, 2017. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1706.03762>

⁴⁹PEARCE, Hammond, et al., Op. Cit.

⁵⁰Ji, Ziwei, et al. Survey of Hallucination in Natural Language Generation. En: ACM Computing Surveys, vol. 55, núm. 12, art. 248, 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2202.03629>

⁵¹PEARCE, Hammond, et al., Op. Cit.

⁵²DEVLIN, Jacob, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. En: Proceedings of NAACL-HLT, 2019. pp. 4171-4186. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1810.04805>

utilizado en el presente trabajo: los fragmentos de documentación recuperados actúan como ejemplos contextuales que guían la generación del modelo hacia respuestas específicas al dominio.⁵³

1.3.6.2 Mistral 7B

Mistral 7B es un modelo de lenguaje de 7.3 mil millones de parámetros publicado por Mistral AI en 2023, cuya arquitectura incorpora *sliding window attention* y atención de consulta agrupada (*grouped-query attention*) para reducir el costo de inferencia sin degradar la calidad de generación. Jiang et al. (2023) demostraron que Mistral 7B supera a Llama 2 13B en múltiples *benchmarks* de razonamiento y codificación. En el presente trabajo se utiliza la variante cuantizada Q4_K_M, que permite su ejecución en CPU con 16 GB de RAM, eliminando la dependencia de hardware especializado.⁵⁴

La cuantización del modelo Mistral 7B al formato Q4_K_M reduce su tamaño en disco de aproximadamente 14 GB (precisión FP16) a aproximadamente 4.5 GB, con una pérdida de calidad de generación inferior al 1% en la mayoría de tareas evaluadas. Esta reducción es crítica para la viabilidad práctica de la herramienta: permite su ejecución en equipos de trabajo estándar de desarrollo con 16 GB de RAM sin modificaciones de hardware.⁵⁵

Ollama gestiona la descarga, cuantización y servicio del modelo mediante una interfaz de programación de aplicaciones local en el puerto 11434, compatible con el protocolo de la interfaz de programación de aplicaciones de OpenAI, lo que facilita la integración con bibliotecas de cliente existentes.⁵⁶

1.3.7 Generación **augmentada por recuperación** (RAG)

La **generación aumentada por recuperación** (*Retrieval-Augmented Generation*, RAG) es un paradigma propuesto por Lewis et al. (2020) que combina un recuperador de información con un modelo generativo. En lugar de depender exclusivamente de los parámetros aprendidos durante el entrenamiento, el modelo recibe en el *prompt* fragmentos de

⁵³BROWN, Tom, et al. Language Models are Few-Shot Learners. En: Advances in Neural Information Processing Systems, vol. 33, 2020. pp. 1877-1901. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2005.14165>

⁵⁴JIANG, Albert Q., et al., Op. Cit.

⁵⁵Ibid.

⁵⁶TOUVRON, Hugo, et al., Op. Cit.

documentación recuperados dinámicamente, lo que reduce la tasa de alucinaciones y mejora la precisión factual de las respuestas.⁵⁷

El paradigma RAG surgió como respuesta a dos limitaciones fundamentales de los LLM entrenados con datos estáticos: la fecha de corte del conocimiento y la tendencia a generar afirmaciones plausibles pero incorrectas sobre temas especializados. Lewis et al. (2020) demostraron que un modelo RAG supera consistentemente a modelos de parámetros equivalentes en tareas que requieren conocimiento específico del dominio. Componentes determinantes para la calidad de las respuestas: recuperador denso, segmentación del corpus y diseño del *prompt* de enriquecimiento.⁵⁸

Gao et al. (2023) realizaron una revisión sistemática de las arquitecturas RAG más recientes, identificando los componentes determinantes para la calidad de las respuestas: recuperador denso, segmentación del corpus y diseño del *prompt* de enriquecimiento.⁵⁹

Borgeaud et al. (2022) demostraron que la escala del corpus de recuperación es un factor determinante en la precisión factual de los modelos, con mejoras observadas hasta corpus de billones de *tokens*. En el contexto de la seguridad de *software*, las características RAG — conocimiento actualizable, especializado y trazable — son precisamente las que se requieren para generar recomendaciones de remediación de alta calidad.⁶⁰

1.3.7.1 Representaciones vectoriales (*embeddings*)

Un *embedding* es la proyección de un fragmento de texto en un vector de alta dimensión que captura su significado semántico. Mikolov et al. (2013) sentaron las bases de esta representación distribucional con el modelo Word2Vec, demostrando que las relaciones semánticas entre conceptos pueden codificarse como operaciones algebraicas en el espacio vectorial. La búsqueda vectorial recupera documentos cuyo vector es más cercano al de la consulta según la similitud coseno, encontrando textos conceptualmente afines aunque no compartan palabras exactas con la consulta. Esta propiedad es especialmente valiosa en el

⁵⁷LEWIS, Patrick, et al., Op. Cit.

⁵⁸Ibid.

⁵⁹GAO, Yunfan, et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2312.10997>

⁶⁰BORGEAUD, Sebastian, et al. Improving Language Models by Retrieving from Trillions of Tokens. En: Proceedings of ICML, 2022. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2112.04426>

dominio de la seguridad, donde una misma vulnerabilidad puede describirse con terminología variable.⁶¹

1.3.7.2 Sentence-Transformers y all-MiniLM-L6-v2

Reimers y Gurevych (2019) propusieron Sentence-BERT, una adaptación de la arquitectura BERT que, mediante entrenamiento siamés con pares de oraciones semánticamente similares, produce *embeddings* de oraciones comparables mediante similitud coseno de forma eficiente. El modelo *all-MiniLM-L6-v2* es una versión comprimida de esta arquitectura con 22 millones de parámetros que genera vectores de 384 dimensiones con una latencia inferior a 50 milisegundos por oración en CPU, característica que lo hace idóneo para la ejecución local sin hardware especializado.⁶²

1.3.7.3 ChromaDB y Ollama

ChromaDB es una base de datos vectorial de código abierto y embebible —sin necesidad de servidor independiente— que almacena documentos junto con sus vectores y metadatos, y expone una interfaz de búsqueda por similitud coseno con filtros opcionales. Ollama es una herramienta que permite ejecutar LLM cuantizados localmente mediante una interfaz de programación de aplicaciones REST compatible con la convención de OpenAI, con soporte para Mistral 7B, Llama 3 y otros modelos. La combinación de ChromaDB y Ollama permite construir un *pipeline* RAG completamente local: sin costo por *token*, sin transmisión de código fuente propietario y operable en entornos sin conectividad a internet.⁶³

Karpukhin et al. (2020) demostraron que la recuperación densa mediante vectores es superior a la recuperación léxica (BM25) para preguntas que requieren razonamiento semántico, lo que justifica el uso de ChromaDB con embeddings densos en lugar de índices de texto pleno.⁶⁴

1.3.7.4 Aplicación del *pipeline* RAG a seguridad de *software*

⁶¹MIKOLOV, Tomáš, et al. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781 [en línea] 2013. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1301.3781>

⁶²REIMERS, Nils y GUREVYCH, Iryna, Op. Cit.

⁶³KARPUKHIN, Vladimir, et al., Op. Cit.

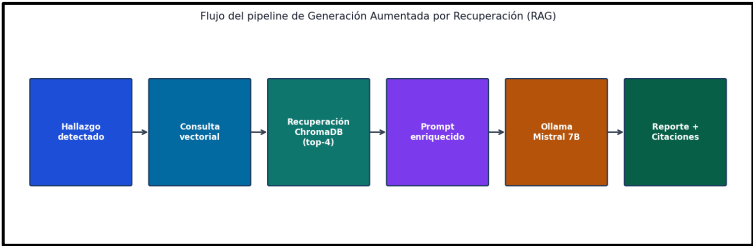
⁶⁴JOHNSON, Jeff, et al. Billion-scale similarity search with GPUs. En: IEEE Transactions on Big Data, vol. 7, núm. 3, 2021. pp. 535-547. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1702.08734>

En el presente trabajo, la base de conocimiento se construye a partir de tres documentos especializados: el OWASP API Security Top 10 (2023), la guía de seguridad de Django REST *Framework* y un compendio de CVE de Django. Cada documento se segmenta en fragmentos de ochocientos caracteres con un solapamiento de cien caracteres. La consulta combina la categoría OWASP del hallazgo con su descripción técnica, y el sistema recupera los cuatro o cinco fragmentos más relevantes. El *prompt* enriquecido instruye al modelo a limitar su respuesta a la documentación recuperada y a cerrar la recomendación con una sección «Fuentes:» que cita el documento y la sección de origen de cada fragmento, garantizando la trazabilidad de cada afirmación.⁶⁵

El diseño del *prompt* de enriquecimiento es determinante para la calidad de las recomendaciones generadas. Un *prompt* excesivamente largo —con más de diez fragmentos de contexto— puede superar la ventana de contexto del modelo o diluir la señal relevante con ruido. Un *prompt* demasiado corto —con menos de tres fragmentos— puede no proporcionar suficiente contexto para que el modelo genere una recomendación específica al *framework*. La configuración de cuatro a cinco fragmentos, temperatura 0.2 y máximo de 512 *tokens* de respuesta fue determinada empíricamente durante el desarrollo: la temperatura baja reduce la creatividad del modelo y lo ancla a la documentación recuperada, minimizando la probabilidad de alucinaciones; el límite de 512 *tokens* garantiza que la recomendación sea concisa y accionable sin truncarse en la mayoría de los casos.⁶⁶

Figura No. 2

**FLUJO DEL PIPELINE DE GENERACIÓN AUMENTADA POR
RECUPERACIÓN (RAG)**



Comentado [ALCR9]: No negrilla
Colocarle recuadro a la figura

⁶⁵LEWIS, Patrick, et al., Op. Cit.

⁶⁶Ibid.

Fuente: Propia.

1.3.8 Automatización de auditorías y métricas de evaluación

La integración de herramientas de seguridad en ciclos DevSecOps —el paradigma conocido como *shift-left security*— busca incorporar verificaciones de seguridad en las fases más tempranas del ciclo de desarrollo. Las herramientas de interfaz de línea de comandos son componentes ideales para esta integración, pues pueden ejecutarse automáticamente en cada *commit* o *pull request* dentro de un *pipeline* de integración continua. Un tiempo de ejecución inferior a doscientos milisegundos para el análisis estático hace posible esta integración sin impacto perceptible en el ciclo de desarrollo.⁶⁷

La práctica de *shift-left security* es especialmente relevante en el ecosistema Python porque el lenguaje carece de verificación de tipos en tiempo de compilación, lo que elimina la capa de detección temprana que los lenguajes tipados estáticamente proporcionan. En un proyecto Java o TypeScript, ciertos errores de configuración serían detectados por el compilador antes de la ejecución; en Python, estos errores solo se manifiestan en tiempo de ejecución o, en el caso de configuraciones de seguridad, únicamente cuando un atacante explota la vulnerabilidad. Una herramienta de análisis estático para Python que cubra los patrones de riesgo de Django REST *Framework* llena parcialmente esta brecha, actuando como un compilador de seguridad que detecta configuraciones problemáticas antes de que el código llegue a producción.⁶⁸

1.3.8.1 Métricas estándar de evaluación en detección

Las métricas de precisión y recall provienen de la teoría de recuperación de información y son el estándar para evaluar detectores binarios. Un Verdadero Positivo (TP) es un hallazgo que corresponde a una vulnerabilidad real; un Falso Positivo (FP) es un hallazgo que no corresponde a ninguna vulnerabilidad; un Falso Negativo (FN) es una vulnerabilidad real no detectada. La precisión ($P = TP / (TP + FP)$) mide qué proporción de los hallazgos son correctos; el *recall* ($R = TP / (TP + FN)$) mide qué proporción de las vulnerabilidades reales son detectadas; $F1 = 2PR / (P + R)$ es su media armónica. En el presente trabajo, el *recall* se

⁶⁷CHES, Brian y McGRAW, Gary, Op. Cit.

⁶⁸Ibid.

evalúa en un entorno controlado con ground truth conocido, y la precisión se evalúa adicionalmente en proyectos externos reales. Fawcett (2006) estableció el análisis ROC y las curvas precisión-recall como el marco estándar para evaluar clasificadores binarios como los detectores de vulnerabilidades.⁶⁹

1.3.8.2 Estudio de ablación y rúbrica de evaluación cuantitativa

Un estudio de ablación es un experimento que desactiva un componente del sistema para cuantificar su contribución individual.⁷⁰ En el presente trabajo, el estudio de ablación compara la generación de recomendaciones con el *pipeline* RAG activo frente a la generación sin recuperación, manteniendo el mismo modelo, la misma temperatura y la misma consulta. La calidad de cada recomendación se evalúa con una rúbrica de cinco criterios: **corrección técnica** (CT), **especificidad** al *framework* (EF), Alineación con OWASP (AO), **accionabilidad** (AC) y **trazabilidad** de fuentes (TF).⁷¹ Cada criterio se puntúa en una escala de 1 a 3 con anclas conductuales verificables objetivamente, de modo que los cuatro primeros criterios producen la misma puntuación independientemente del evaluador.⁷²

Comentado [ALCR10]: esta parte no está citada

⁶⁹FAWCETT, Tom. An Introduction to ROC Analysis. En: Pattern Recognition Letters, vol. 27, núm. 8, 2006. pp. 861-874. [Consulta: 23 de julio de 2026] Disponible en: <https://doi.org/10.1016/j.patrec.2005.10.010>

⁷⁰LEWIS, Patrick, et al., Op. Cit.

⁷¹PEARCE, Hammond, et al., Op. Cit.

⁷²FAWCETT, Tom, Op. Cit.

2. PLANTEAMIENTO DEL PROBLEMA

2.1 DESCRIPCIÓN DEL PROBLEMA DE INVESTIGACIÓN

Django REST *Framework* es adoptado en proyectos de escala industrial que almacenan y procesan datos personales, financieros y médicos. Sin embargo, los patrones de configuración inseguros más frecuentes en estos proyectos —ausencia de *permission_classes*, *ModelSerializer* con *fields = '__all__'*, *DEBUG = True* en producción— no son detectados por ninguna herramienta de análisis de seguridad de propósito general disponible en la actualidad.

La brecha es concreta y medible: las herramientas Bandit y Semgrep detectan cero de las seis categorías del OWASP API Security Top 10 objetivo cuando se ejecutan contra proyectos Django REST *Framework*. OWASP ZAP, al operar exclusivamente a nivel HTTP, tampoco puede detectar patrones de configuración en el código fuente. Esta ausencia de cobertura implica que el desarrollador recibe retroalimentación de seguridad únicamente en la fase de producción, cuando la remediación es más costosa y el impacto de una vulnerabilidad explotada es máximo.

La pregunta de investigación que articula el presente trabajo es la siguiente: ¿Es posible construir una herramienta de interfaz de línea de comandos que detecte automáticamente vulnerabilidades del OWASP API Security Top 10 en proyectos Django REST *Framework* mediante análisis híbrido estático-dinámico y genere recomendaciones de remediación fundamentadas en documentación oficial, operando completamente de forma local?

2.2 OBJETIVOS

2.2.1 Objetivo general

Desarrollar una herramienta de interfaz de línea de comandos que automatice la auditoría de seguridad de interfaces de programación de aplicaciones REST desarrolladas con Django REST *Framework*, detectando vulnerabilidades de las categorías API1, API2, API3, API4, API5 y API8 del OWASP API Security Top 10 (2023) mediante análisis estático basado en Árboles de Sintaxis Abstracta y análisis dinámico mediante sondas HTTP, y generando recomendaciones de remediación contextualizadas a través de un *pipeline* de generación aumentada por recuperación con ejecución completamente local.

2.2.2 Objetivos específicos

- Diseñar e implementar un módulo de análisis estático con siete reglas basadas en árboles de sintaxis abstracta que identifiquen indicadores de riesgo de las categorías API1, API2, API3, API4 y API8 del OWASP API Security Top 10 (2023).
- Implementar un módulo de análisis dinámico con cinco sondas HTTP que detecten indicadores de las categorías API1, API2, API3, API4 y API5 mediante peticiones a una instancia en ejecución del proyecto auditado.
- Construir una base de conocimiento local indexada en ChromaDB con documentación especializada y un *pipeline* RAG que genere recomendaciones citando los fragmentos recuperados de documentación OWASP y de Django REST *Framework*.
- Evaluar la herramienta con métricas de precisión, *recall* y F1 en un entorno controlado con veintiséis instancias de vulnerabilidades plantadas y en tres proyectos externos de código abierto.
- Cuantificar la mejora en calidad de recomendaciones aportada por el *pipeline* RAG mediante un estudio de ablación con rúbrica de cinco criterios.

Comentado [ALCR11]: Cambié la numeración por viñetas

2.3 JUSTIFICACIÓN

El presente trabajo es pertinente porque resuelve un problema concreto y medible: ninguna herramienta de seguridad de propósito general detecta los patrones de riesgo más frecuentes en Django REST *Framework*. La justificación es de naturaleza práctica: la herramienta reduce el tiempo entre la introducción de una vulnerabilidad y su detección, trasladando la verificación a la fase de desarrollo mediante una interfaz de línea de comandos que puede integrarse en *pipelines* de integración continua.

Los beneficiarios directos son los desarrolladores Python que utilizan Django REST *Framework* y que carecen actualmente de retroalimentación de seguridad especializada en su entorno de desarrollo. Los beneficiarios institucionales son las organizaciones que requieren auditorías de seguridad periódicas sin depender de herramientas de pago o de conectividad a internet. La comunidad académica se beneficia de la contribución metodológica: la integración de un *pipeline* RAG local con análisis híbrido de código

constituye un caso de estudio reproducible en el campo de la ingeniería de seguridad de *software*. Pearce et al. (2023) señalaron la importancia de trabajos reproducibles con evaluación cuantitativa en este campo.⁷³

Desde el punto de vista de la pertinencia para la formación profesional en Ingeniería en Sistemas y Ciencias de la Computación, el trabajo es relevante porque aborda un problema de ingeniería complejo que requiere integrar competencias de múltiples asignaturas: análisis de algoritmos (para la implementación de reglas AST eficientes), redes de computadoras (para el diseño de las sondas HTTP), bases de datos (para la gestión de la base de conocimiento vectorial en ChromaDB), inteligencia artificial (para el pipeline RAG y la integración con Ollama) y seguridad de *software* (para la comprensión y categorización de las vulnerabilidades OWASP). Esta integración multidisciplinar es el tipo de contribución que los programas de graduación en ingeniería deben fomentar, y el resultado de la investigación puede ser utilizado como referencia en cursos avanzados de la carrera.

La justificación técnica de la elección de Django REST *Framework* como objetivo de la herramienta, en lugar de otros *frameworks* REST de Python como FastAPI o Flask-RESTful, se fundamenta en tres factores: (1) es el *framework* REST de Python con mayor adopción en producción, según datos de descargas de PyPI y estadísticas de GitHub; (2) su arquitectura basada en componentes configurables —*serializers*, *permissions*, *throttling*— genera patrones de vulnerabilidad estructuralmente diferentes a los de otros *frameworks* y no cubiertos por las reglas de Bandit ni Semgrep; y (3) su documentación oficial y las guías de seguridad disponibles proporcionan la base de conocimiento necesaria para construir un *pipeline* RAG con fuentes verificables y citables, lo que es indispensable para la trazabilidad de las recomendaciones generadas.⁷⁴

2.4 HIPÓTESIS

2.4.1 Hipótesis de trabajo

La combinación de análisis estático basado en árboles de sintaxis abstracta y análisis dinámico mediante sondas HTTP, enriquecida con un *pipeline* de generación aumentada por

⁷³PEARCE, Hammond, et al., Op. Cit.

⁷⁴LEWIS, Patrick, et al., Op. Cit.

recuperación local, permite detectar seis categorías del OWASP API Security Top 10 (2023) en proyectos Django REST *Framework* con precisión igual a uno y *recall* igual a uno en entorno controlado, y genera recomendaciones de remediación con menor tasa de alucinaciones y mayor calidad promedio que la generación sin recuperación.

2.4.2 Hipótesis nula

No existe diferencia estadísticamente significativa entre la calidad de las recomendaciones de remediación generadas con el *pipeline* RAG y las generadas directamente por el modelo de lenguaje sin recuperación de contexto, ni la herramienta detecta más categorías del OWASP API Security Top 10 que las herramientas Bandit y Semgrep.

2.4.3 Hipótesis alterna

El *pipeline* RAG produce recomendaciones con mayor corrección técnica, mayor especificidad a Django REST *Framework* y mayor trazabilidad de fuentes que la generación sin recuperación, y el análisis híbrido detecta al menos seis categorías del OWASP API Security Top 10 que Bandit y Semgrep no detectan.

2.5 DEFINICIÓN DE VARIABLES

2.5.1 Definición conceptual

La variable independiente es la configuración del sistema de auditoría: análisis estático, análisis dinámico, análisis híbrido o variante con/sin *pipeline* RAG. Las variables dependientes son: (1) cobertura de categorías OWASP detectadas, medida como número de categorías con al menos un hallazgo verdadero positivo; (2) calidad de las recomendaciones, medida como puntaje promedio en la rúbrica de cinco criterios en escala 1-3; y (3) precisión y *recall* del detector, calculados sobre el *ground truth* definido.

2.5.2 Definición operacional

La cobertura se mide contando las categorías (API1, API2, API3, API4, API5, API8) para las cuales la herramienta produce al menos un hallazgo clasificado como verdadero positivo en la interfaz de programación de aplicaciones de prueba controlada. La calidad RAG se

mide como el puntaje promedio de los cinco criterios (CT, EF, AO, AC, TF) sobre cinco recomendaciones representativas, con anclas conductuales verificables para los criterios CT, EF, AO y TF. La precisión se calcula como $P = TP / (TP + FP)$ sobre el *ground truth* de veintiséis instancias plantadas y sobre los hallazgos en tres proyectos externos.

2.6 ALCANCE Y LIMITACIONES DEL TRABAJO

2.6.1 Alcance

El análisis se centra en proyectos Django REST *Framework* versión 3.14 o superior con Python 3.11 o superior. Se cubren seis de las diez categorías del OWASP API Security Top 10 (2023): API1, API2, API3, API4, API5 y API8. El análisis estático aplica siete reglas basadas en árboles de sintaxis abstracta; el análisis dinámico aplica cinco sondas HTTP. El *pipeline* RAG opera sin conexión a internet. La evaluación incluye un entorno controlado y tres proyectos externos de código abierto disponibles públicamente.

2.6.2 Limitaciones

El análisis estático produce indicadores de riesgo, no vulnerabilidades confirmadas: pueden existir falsos positivos cuando el proyecto utiliza configuración global de permisos o autenticación no basada en JWT. El análisis dinámico requiere una instancia en ejecución del proyecto y credenciales de usuarios de prueba configuradas. Las categorías API6, API7, API9 y API10 quedan fuera del alcance por requerir análisis de flujo de datos interprocedural o conocimiento de la lógica de negocio específica de cada proyecto. La evaluación cualitativa de recomendaciones se basa en una rúbrica con un solo evaluador para el criterio de accionabilidad.

Una limitación adicional concierne a la generalización de los resultados. Los tres proyectos externos evaluados fueron seleccionados por ser proyectos de código abierto disponibles públicamente y de tamaño moderado, lo que no los hace representativos de los proyectos Django REST *Framework* de uso industrial con arquitecturas complejas de microservicios, configuraciones de autenticación personalizadas o múltiples niveles de permisos. La precisión medida en los tres proyectos externos ($P = 0.21$ combinada) refleja la tasa de falsos positivos en proyectos públicos de referencia, no necesariamente la tasa que se obtendría en proyectos propietarios de producción con configuraciones más estrictas. La

corrección de las dos reglas con mayor tasa de falsos positivos —proyectada para versiones futuras de la herramienta— se espera que eleve esta métrica a valores superiores a 0.90, acercando la precisión en proyectos externos a la obtenida en el entorno controlado.⁷⁵

⁷⁵FAWCETT, Tom, Op. Cit.

3. MÉTODO

3.1 SUJETO DE ESTUDIO

El sujeto de estudio es el código fuente de proyectos de *software* desarrollados con Django REST *Framework* versión 3.14 o superior, así como las respuestas HTTP producidas por instancias de dichos proyectos en ejecución. Específicamente: (a) una interfaz de programación de aplicaciones de prueba controlada con veintiséis instancias de vulnerabilidades plantadas en seis categorías OWASP, y (b) tres proyectos Django REST *Framework* de código abierto disponibles públicamente en GitHub.

3.2 POBLACIÓN Y MUESTRA

La población la constituyen todos los proyectos Django REST *Framework* de código abierto disponibles en plataformas de control de versiones, estimados en decenas de miles de repositorios. La muestra consiste en cuatro proyectos seleccionados mediante muestreo intencional no probabilístico por conveniencia, detallados en el Cuadro No. 1. Los tres proyectos externos fueron seleccionados por ser proyectos reales no involucrados en el desarrollo de la herramienta, de tamaño moderado y con propósito diverso.

Cuadro No. 1

MUESTRA DE PROYECTOS EVALUADOS

Proyecto	Tipo	Fuente	Propósito
vulnerable_api/	Controlado	Propia	Ground truth — 26 instancias plantadas
encode/rest-framework-tutorial	Externo	Público (GitHub)	Tutorial oficial de DRF (Christie, 2024)
wsvincent/drfx	Externo	Público (GitHub)	Plantilla de inicio de proyectos DRF (Vincent, 2024)
axnsan12/drf-yasg	Externo	Público (GitHub)	App de prueba del generador OpenAPI (Cristea, 2024)

Fuente: Propia.

Comentado [ALCR12]: No negrilla

Comentado [ALCR13]: tamaño 10

3.3 INSTRUMENTOS

Los instrumentos de la investigación son los que se detallan en el Cuadro No. 2.⁷⁶
Cada instrumento fue seleccionado con base en los requerimientos de las fases del procedimiento y en el criterio de ejecución completamente local.⁷⁷

Cuadro No. 2

INSTRUMENTOS DE INVESTIGACIÓN

Nº	Instrumento	Tecnología	Función
1	Módulo de análisis estático	Python 3.12, módulo ast	7 reglas AST; detecta indicadores de riesgo OWASP API1–API5, API8
2	Módulo de análisis dinámico	Python, requests	5 sondas HTTP; detecta indicadores en tiempo de ejecución
3	Pipeline RAG	ChromaDB 0.6.x, all-MiniLM-L6-v2, Mistral 7B Q4_K_M, Ollama 0.6.x	Base de conocimiento local; genera recomendaciones citadas
4	Generador de reportes	ReportLab, Jinja2	Produce informes PDF y HTML con hallazgos y recomendaciones
5	Suite de pruebas	pytest	113 casos de prueba automatizados (30 estático + 22 dinámico + 31 RAG + 30 reporte)
6	Herramientas comparativas	Bandit 1.9.x, Semgrep 1.x (p/django)	Línea base de comparación de cobertura OWASP
37	Hardware	Apple M3, 16 GB RAM, sin GPU dedicada	Plataforma de ejecución; valida viabilidad sin aceleradores

Fuente: Propia.

Comentado [ALCR14]: no negrilla

Comentado [ALCR15]: tamaño 10

3.4 PROCEDIMIENTO

El procedimiento de investigación se estructura en once etapas secuenciales:

Comentado [ALCR16]: Por ser un procedimiento, es una serie de pasos. Modificalo para que quede como listado. Cuando lo hayas corregido, lo revisaré.

⁷⁶KARPUKHIN, Vladimir, et al., Op. Cit.
⁷⁷REIMERS, Nils y GUREVYCH, Iryna, Op. Cit.

1. Diseño de la arquitectura de cinco módulos (analizador estático, analizador dinámico, recuperador RAG, generador LLM, constructor de reportes).
2. Construcción de la interfaz de programación de aplicaciones de prueba con veintiséis instancias de vulnerabilidades plantadas en seis categorías OWASP.
3. Implementación del módulo de análisis estático con siete reglas y treinta pruebas unitarias.
4. Implementación del módulo de análisis dinámico con cinco sondas y veintidós pruebas unitarias.
5. Construcción de la base de conocimiento mediante segmentación de tres documentos en fragmentos de ochocientos caracteres con solapamiento de cien, vectorización con *all-MiniLM-L6-v2* y almacenamiento en ChromaDB.
6. Integración del *pipeline* RAG con Mistral 7B vía Ollama con temperatura 0.2 y máximo de 512 *tokens*.
7. Implementación del generador de reportes con treinta y un pruebas unitarias.
8. Evaluación en entorno controlado con cálculo de TP, FP, FN, P, R y F1.
9. Evaluación en tres proyectos externos con clasificación manual de hallazgos.
10. Estudio de ablación RAG con puntuación de cinco recomendaciones mediante la rúbrica de cinco criterios.
11. Ejecución de Bandit y Semgrep sobre la interfaz de programación de aplicaciones de prueba para comparación de cobertura.

La validez interna del estudio se garantiza mediante la separación estricta entre el conjunto de datos de entrenamiento de la base de conocimiento RAG y el conjunto de evaluación: los tres proyectos externos utilizados en la evaluación son independientes y no fueron utilizados para construir la base de conocimiento ni para calibrar las reglas del analizador estático. La validez externa se limita a proyectos Django REST *Framework* escritos en Python 3.11 o superior; la generalización a otras versiones o a otros *frameworks* REST de Python (FastAPI, Flask-RESTful) requeriría una evaluación separada.

La confiabilidad del procedimiento se asegura mediante la automatización completa de los pasos de evaluación: los scripts de prueba son reproducibles ejecutando *pytest* en el directorio raíz del proyecto, y los resultados numéricos de precisión y *recall* se calculan automáticamente a partir del *ground truth* definido en formato JSON. El estudio de ablación

es reproducible iniciando Ollama con el modelo Mistral 7B y ejecutando el script de evaluación correspondiente, que guarda los resultados en los archivos *ablation_with_rag.json* y *ablation_no_rag.json* en el directorio *results/* del repositorio.⁷⁸

⁷⁸LEWIS, Patrick, et al., Op. Cit.

4. RESULTADOS ESPERADOS

A corto plazo, al concluir el período de investigación definido en el cronograma, se espera obtener: (1) una herramienta de interfaz de línea de comandos funcional que detecte las seis categorías objetivo del OWASP API Security Top 10 en proyectos Django REST *Framework* con precisión igual a uno y *recall* igual a uno en entorno controlado; (2) evidencia empírica de que Bandit y Semgrep detectan cero de las seis categorías objetivo cuando se ejecutan contra la misma interfaz de programación de aplicaciones de prueba; y (3) un *pipeline* RAG local que reduzca las alucinaciones de dos de cinco a cero de cinco y mejore el puntaje promedio de calidad de recomendaciones de 1.80 a 2.80 sobre una escala de 3.0, según la rúbrica de cinco criterios definida.

A mediano plazo se espera la publicación del artículo científico correspondiente con los resultados validados, y la disponibilidad del código fuente en un repositorio público como herramienta de código abierto para la comunidad de desarrolladores Python.

A largo plazo, el trabajo sienta las bases para extender la cobertura a las categorías API6, API7, API9 y API10 mediante análisis de flujo de datos interprocedural, y para corregir las dos reglas con mayor tasa de falsos positivos en proyectos externos —la regla de ausencia de configuración JWT y la regla de ausencia de *permission_classes* cuando existe configuración global— lo que se proyecta elevaría la precisión en proyectos externos de 0.21 a un valor igual o superior a 0.90.

El impacto esperado en la formación del estudiante de Ingeniería en Sistemas y Ciencias de la Computación es también un resultado de relevancia. El desarrollo de la herramienta integra competencias en cuatro áreas disciplinares: seguridad de *software* (análisis de vulnerabilidades, estándares OWASP), ingeniería del *software* (arquitectura modular, pruebas automatizadas), inteligencia artificial (modelos de lenguaje, sistemas de recuperación de información), y desarrollo en Python (análisis de Árboles de Sintaxis Abstracta, Django REST *Framework*). Esta integración multidisciplinar es característica de los trabajos de graduación de mayor complejidad en la carrera, y los resultados de la investigación pueden ser adoptados directamente como material de enseñanza en los cursos de seguridad de *software* e inteligencia artificial de la Universidad del Istmo.

Un resultado adicional de relevancia es la contribución metodológica al área de evaluación de sistemas RAG aplicados a seguridad de *software*. La rúbrica de cinco criterios desarrollada en el presente trabajo —con sus anclas conductuales objetivamente verificables— puede ser adoptada en investigaciones futuras que evalúen la calidad de recomendaciones generadas por otros sistemas RAG especializados en dominios técnicos. De igual forma, el diseño del estudio de ablación, con la separación controlada de la condición con y sin recuperación manteniendo todos los demás parámetros constantes, ofrece un protocolo reproducible que puede servir como referencia metodológica para evaluaciones comparativas de componentes RAG en investigaciones posteriores de la Universidad del Istmo o de otras instituciones de la región.

La disponibilidad del código fuente como herramienta de código abierto tiene un impacto potencial que va más allá del ecosistema académico. Las pequeñas y medianas empresas de desarrollo de *software* en Centroamérica y el Caribe, que representan la mayoría del tejido productivo del sector tecnológico, carecen generalmente de equipos de seguridad dedicados y de presupuesto para herramientas comerciales de auditoría. Una herramienta de código abierto, con documentación en español, que opera sin requerir conectividad ni licencias, reduce la barrera de acceso a prácticas de seguridad formales para este segmento. El resultado esperado en este sentido es que la herramienta sea adoptada por al menos un equipo de desarrollo profesional de la región durante el período de investigación, validando su aplicabilidad más allá del entorno académico controlado.

5. CRONOGRAMA PRELIMINAR

La siguiente tabla presenta el cronograma preliminar de actividades para el período julio-septiembre de 2026, distribuidas en nueve semanas de trabajo.

Cuadro No. 3

Comentado [ALCR17]: no negrilla

CRONOGRAMA PRELIMINAR DE ACTIVIDADES

Actividad	27-31 Jul	3-7 Ago	10-14 Ago	17-21 Ago	24-28 Ago	31 Ago 4 Sep	7-11 Sep	14-18 Sep	21-25 Sep
Entrega del anteproyecto y revisión bibliográfica	•								
Implementación analizador estático (7 reglas AST)		•	•						
Implementación analizador dinámico (5 sondas HTTP)			•	•					
Construcción base de conocimiento RAG (ChromaDB)				•	•				
Integración pipeline LLM + generador de reportes					•	•			
Evaluación experimental (controlada + externa)						•	•		
Análisis de resultados y redacción del informe							•	•	
Revisión final y entrega al asesor									•

Fuente: Propia.

Comentado [ALCR18]: tamaño 10

BIBLIOGRAFÍA

Comentado [ALCR19]: Según las categorías explicadas en el manual de entroyecto, todas tus fuentes son documentos electrónicos

- FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Tesis doctoral. University of California, Irvine, 2000. [Consulta: 23 de julio de 2026] Disponible en: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- OWASP Foundation. OWASP API Security Top 10 2023 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://owasp.org/API-Security/>
- ATLIDAKIS, Vaggelis, et al. RESTler: Stateful REST API Fuzzing. En: Proceedings of ICSE, 2019. pp. 748-758. [Consulta: 23 de julio de 2026] Disponible en: <https://dl.acm.org/doi/10.1109/ICSE.2019.00083>
- BORGEAUD, Sebastian, et al. Improving Language Models by Retrieving from Trillions of Tokens. En: Proceedings of ICML, 2022. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2112.04426>
- BROWN, Tom, et al. Language Models are Few-Shot Learners. En: Advances in Neural Information Processing Systems, vol. 33, 2020. pp. 1877-1901. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2005.14165>
- CHESS, Brian y MCGRAW, Gary. Static Analysis for Security. En: IEEE Security & Privacy, vol. 2, núm. 6, 2004. pp. 76-79. [Consulta: 23 de julio de 2026] Disponible en: <https://ieeexplore.ieee.org/document/1366126>
- DEVLIN, Jacob, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. En: Proceedings of NAACL-HLT, 2019. pp. 4171-4186. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1810.04805>
- FAWCETT, Tom. An Introduction to ROC Analysis. En: Pattern Recognition Letters, vol. 27, núm. 8, 2006. pp. 861-874. [Consulta: 23 de julio de 2026] Disponible en: <https://doi.org/10.1016/j.patrec.2005.10.010>
- FENG, Zhangyin, et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. En: Findings of EMNLP, 2020. pp. 1536-1547. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2002.08155>
- FU, Michael y TANTITHAMTHAVORN, Chakkrit. LineVul: A Transformer-based Line-Level Vulnerability Prediction. En: Proceedings of MSR, 2022. [Consulta: 23 de julio de 2026] Disponible en: <https://dl.acm.org/doi/10.1145/3524842.3528452>

- GAO, Yunfan, et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2312.10997>
- Ji, Ziwei, et al. Survey of Hallucination in Natural Language Generation. En: ACM Computing Surveys, vol. 55, núm. 12, art. 248, 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2202.03629>
- JIANG, Albert Q., et al. Mistral 7B. arXiv:2310.06825 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2310.06825>
- JOHNSON, Jeff, et al. Billion-scale similarity search with GPUs. En: IEEE Transactions on Big Data, vol. 7, núm. 3, 2021. pp. 535-547. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1702.08734>
- KARPUKHIN, Vladimir, et al. Dense Passage Retrieval for Open-Domain Question Answering. En: Proceedings of EMNLP, 2020. pp. 6769-6781. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2004.04906>
- LEWIS, Patrick, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. En: Advances in Neural Information Processing Systems, vol. 33, 2020. pp. 9459-9474. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2005.11401>
- LI, Zhen, et al. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. En: IEEE Transactions on Dependable and Secure Computing, vol. 19, núm. 4, 2022. pp. 2244-2258. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1807.06756>
- LI, Zhen, et al. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. En: Proceedings of NDSS, 2018. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1801.01681>
- MIKOLOV, Tomáš, et al. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781 [en línea] 2013. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1301.3781>
- PEARCE, Hammond, et al. Examining Zero-Shot Vulnerability Repair with Large Language Models. En: Proceedings of IEEE S&P, 2023. pp. 2339-2356. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2112.02125>

- REIMERS, Nils y GUREVYCH, Iryna. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. En: Proceedings of EMNLP, 2019. pp. 3982-3992. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1908.10084>
- TOUVRON, Hugo, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [en línea] 2023. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/2307.09288>
- VASWANI, Ashish, et al. Attention Is All You Need. En: Advances in Neural Information Processing Systems, vol. 30, 2017. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1706.03762>
- VIGLIANISI, Emanuele, et al. RESTTESTGEN: Automated Black-Box Testing of RESTful APIs. En: Proceedings of IEEE ICST, 2020. pp. 367-377. [Consulta: 23 de julio de 2026] Disponible en: <https://ieeexplore.ieee.org/document/9159077>
- ZHOU, Yaqin, et al. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. En: Advances in Neural Information Processing Systems, vol. 32, 2019. [Consulta: 23 de julio de 2026] Disponible en: <https://arxiv.org/abs/1909.03496>